

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Participación sobre continuaciones lenguajes

PRESENTA

Jiménez Rivera Emiliano Kaleb

PROFESOR

Manuel Soto Romero

ASIGNATURA

Lenguajes de Programación

November 12, 2025

Como hemos visto en clase sabemos que el cálculo lambda es turing completo, es decir que se puede representar todo algoritmo computable solo con variables, abstracciones y aplicaciones de funciones.

Uno de los algoritmos de conversion a kontinuaciones es el conocido como el algoritmo de Danvy-Filinski para CPS. Su funcionamiento radica en ir instruccin por instruccin de un programa (en este caso funcional) e ir transformándolo poco a poco a CPS dependiendo del tipo de valor que "lea", pues va expresión por expresión convirtiendo para llegar a una versión final de CPS.

Una observación que quisiera agregar es que implícitamente este algoritmo se encuentra en algunas de las formalizaciones del análisis semántico de las kontinuaiones en nuestro MiniLisp, pues la forma en que se reducen es muy similar.

[1] Valores (números, booleanos, variables)

$$\text{CPS}[\![v]\!] k = k v$$

[2] Operaciones binarias

$$\text{CPS}[\![e_1 \oplus e_2]\!] k = \text{CPS}[\![e_1]\!] (\lambda v_1. \text{CPS}[\![e_2]\!] (\lambda v_2. k (v_1 \oplus v_2)))$$

[3] Condicionales

$$\text{CPS}[\![\text{if } e \text{ then } e_1 \text{ else } e_2]\!] k = \text{CPS}[\![e]\!] (\lambda v. \text{if } v \text{ then } \text{CPS}[\![e_1]\!] k \text{ else } \text{CPS}[\![e_2]\!] k)$$

[4] Funciones lambda

$$\text{CPS}[\![\lambda x. e]\!] k = k (\lambda x. \lambda k'. \text{CPS}[\![e]\!] k')$$

[5] Aplicación de función

$$\text{CPS}[\![e_1 e_2]\!] k = \text{CPS}[\![e_1]\!] (\lambda v_1. \text{CPS}[\![e_2]\!] (\lambda v_2. v_1 v_2 k))$$

[6] Let

$$\text{CPS}[\![\text{let } x = e_1 \text{ in } e_2]\!] k = \text{CPS}[\![e_1]\!] (\lambda v. \text{CPS}[\![e_2[x := v]]\!] k)$$

1 Ejemplo de uso

Factorial Original

```
fact n = if (n == 0) then 1 else n * fact (n-1)
```

Transformación a CPS

Expandir definición:

$$\begin{aligned} & \text{CPS}[\![\text{fact } n]\!] k \\ &= \text{CPS}[\![\text{if } (n == 0) \text{ then } 1 \text{ else } n * \text{fact}(n - 1)]\!] k \end{aligned}$$

Aplicamos regla 3 (condicional):

$$= \text{CPS}[\![n == 0]\!] (\lambda v. \text{if } v \text{ then } \text{CPS}[\![1]\!] k \text{ else } \text{CPS}[\![n * \text{fact}(n - 1)]\!] k)$$

$n == 0$ es comparación (asumamos primitiva), devolverá Bool:

$$= (\lambda v. \text{if } v \text{ then } \text{CPS}[\![1]\!] k \text{ else } \text{CPS}[\![n * \text{fact}(n - 1)]\!] k) (n == 0)$$

Caso 1: $n == 0$ (rama then)

$$\text{CPS}\llbracket 1 \rrbracket k = k \ 1 \quad (\text{regla 1})$$

Caso 2: $n \neq 0$ (rama else)

$$\text{CPS}\llbracket n * \text{fact}(n - 1) \rrbracket k$$

Aplicamos regla 2 (operación binaria):

$$= \text{CPS}\llbracket n \rrbracket (\lambda v_1. \text{CPS}\llbracket \text{fact}(n - 1) \rrbracket (\lambda v_2. k (v_1 * v_2)))$$

n es valor:

$$\begin{aligned} &= (\lambda v_1. \text{CPS}\llbracket \text{fact}(n - 1) \rrbracket (\lambda v_2. k (v_1 * v_2))) n \\ &= \text{CPS}\llbracket \text{fact}(n - 1) \rrbracket (\lambda v_2. k (n * v_2)) \end{aligned}$$

$\text{fact}(n - 1)$ es llamada recursiva:

$$= \text{fact_cps} (n - 1) (\lambda v_2. k (n * v_2))$$

*Resultado Final de nuestro proceso

```
fact_cps n k =
  if (n == 0)
  then k 1
  else fact_cps (n-1) (\lambda v. k (n * v))
```

-- Función principal:

```
fact n = fact_cps n (\lambda x.x)
```

Vemos que es un algoritmo algo complejo de analizar cuando el algoritmo original es recursivo o inclusive muy largo. Algunos datos interesantes sobre este algoritmo es que es utilizado para la creación de compiladores e intérpretes. Compiladores de lenguajes funcionales (como Scheme, ML, Haskell) transforman el código a CPS como representación intermedia antes de generar código máquina, además facilita el flujo de control y optimiza resoluciones de errores. Su gran ventaja es que es completamente explícito.

References

- [1] Appel, A. W. (1992). *Compiling with continuations*. Cambridge University Press.
- [2] Danvy, O., & Filinski, A. (1992). Representing control: A study of the CPS transformation. *Mathematical Structures in Computer Science*, 2(4), 361–391. <https://doi.org/10.1017/S0960129500001535>
- [3] Danvy, O., & Nielsen, L. R. (2003). A first-order one-pass CPS transformation. *Theoretical Computer Science*, 308(1–3), 239–257.